Developing algorithms with the BioPiM Library

E. Drezen[1], F. De Moor[2], M. Mognol[2], D. Lavenier[2]

[1]Pasteur Institute, Paris – [2]CNRS-IRISA, Rennes
France

February 2026

1 Introduction

Designing an efficient algorithm is often a delicate art, as it requires not only achieving a reasonably low algorithmic complexity but also making the best possible use of the hardware resources available in order to speed up the algorithm's execution.

It is therefore necessary to adopt both the researcher's point of view for the purely algorithmic aspects and the engineer's point of view to find the best possible implementation with respect to the chosen hardware architecture. Moreover, some hardware aspects can have a direct impact on the algorithm itself. One may cite, for example, the literature on cache-friendly algorithms, which attempt to make optimal use of memory caches in architectures featuring L1, L2, and L3 caches. Another example is the PIM architecture from UPMEM, which enables a very high level of parallelism (i.e., tens of thousands of threads running concurrently), sometimes requiring specific considerations regarding how to partition the data processed by the algorithm.

Beyond these considerations, a hardware architecture always comes with a programming model that may be more or less mature and more or less easy for the developer to use. Today, hardware architectures such as multi-core processors and GPU-type graphics cards provide APIs (Application Programming Interfaces) that are widely adopted by the community and allow algorithms to be developed easily. But this is not necessarily the case for more exotic architectures, as they often stem from recent research and therefore lack historical maturity to have converged toward a simple-to-use API. This is, in practice, a crucial point, because an API that is too complex or perceived as too "low-level" can become a psychological barrier for developers who are reluctant to invest in learning a programming model deemed difficult.

1.1 The UPMEM PiM Architecture

In this regard, UPMEM's PIM architecture is an interesting case. Its hardware characteristics can be easily understood by the developer by drawing a parallel with the familiar world of multi-core architectures. At first approximation, one can consider that UPMEM's logical execution unit, called a "tasklet," is equivalent to a "thread" in a multi-core environment, and that an API similar to that used for threads can be employed. Unfortunately, this represents only a small part of the problem, as the developer must take into account

low-level aspects such as communication between the CPUs and the PIM memories called DPUs, specific memory-alignment constraints, and the fact that it is necessary to write not one but two programs: one for the so-called "host" running on the CPUs, and another for the DPU memories. This also impacts the compilation toolchain, since two programs must be generated, each compiled with its own compiler and specific options.

Another characteristic of UPMEM is the existence of several levels of parallelization. In a multi-core context, one may see only a single level, insofar as N threads are created to work in parallel. In the case of UPMEM, there are three levels of parallelization, called rank/DPU/tasklets: one rank controls 64 DPUs, and each DPU controls 16 tasklets, the tasklet being the logical unit that performs computations. As we shall see later, this particularity must be taken into account if one wishes to propose an API close to that used in multi-core environments.

Today, the API provided by UPMEM therefore exhibits intrinsic characteristics of the hardware architecture's very design, which makes learning it and using it on a daily basis rather delicate. The remainder of this document will introduce the BPL (BioPim Library), which not only simplifies development for the UPMEM architecture by adopting a more commonly used programming model—such as the multithreaded multicore model, but also makes it possible to write code that can run both on the UPMEM architecture and on a multicore architecture, and potentially on other architectures in the future.

1.2 Overview of the BioPiM Library

A fundamental design goal of the BPL is to enable writing algorithms in a way that is almost identical to the more conventional approach one would adopt, for example through a multithreaded model on a multicore architecture.

To understand the intended benefit, an example of a simple program[1] shows how to compute the sum of the elements of an array, using the native UPMEM C API on one side (66 lines of code) and the BPL C++ API on the other (18 lines of code).

In the UPMEM case, the following points can be observed:

- the dichotomy between the program intended for the CPU and the one intended for the DPUs is clearly visible due to the use of two separate subsets of functions ("host" API for the CPU and "runtime" API for the DPUs). The developer must therefore genuinely think of their application as two distinct programs, which may feel rather unnatural.

- in a certain sense, the DPU program can be viewed as a function F that takes input parameters and returns a result. The role of the CPU program is to handle the low-level management of data transfers between the CPU and the DPUs to deliver the input parameters to function F, and then to retrieve the function's output. This management resembles what one might encounter when computation occurs on a remote node in a machine cluster, or through a Remote Procedure Call (RPC) mechanism allowing the invocation of a remote procedure.

In the BPL case, we observe:

[1] The source code is presented in the section that defines the LOC metric.

- a clear definition of the algorithm in the form of a function that takes the data as input, computes the result, and returns it as the function's output
- a main function that executes the algorithm through a "launcher" object. This object is created by specifying the intent to run the function on the UPMEM architecture, and this is the only reference to the target architecture: all remaining code resembles "classical" code without any mention of low-level features.

We can already see the value of the BPL in terms of ease of algorithm development. The aspects tied to the characteristics of UPMEM's hardware architecture are hidden inside the concept of the "launcher," which handles all low-level operations. Furthermore, the developer remains in familiar territory and can focus more on the algorithm than on architecture-specific considerations.

Regarding the build system, the BPL library uses the widely adopted CMake tool. A default configuration is provided to the developer, who therefore does not need to worry about writing scripts to generate these programs.

The ambition of the BPL is thus to allow developers to quickly take advantage of the UPMEM architecture without the long learning curve associated with the native UPMEM API. For further details on the BPL, the reader may refer to the user documentation. In the remainder of this document, we will review several algorithms developed for a PIM architecture (1) using the low-level UPMEM SDK and (2) using the BPL; this will allow us to compare the relative performance of the two approaches. Note that in some cases, we will also be able to compare performance using a multicore architecture.

2 Workloads

2.1 Bloom filters library for large batches of items

Presentation BFlib is a Bloom filters library aimed at storing and querying large chunks of elements at once. Instead of one big array, the library uses a collection of smaller filters. The library is specifically designed to run with the Processing-in-Memory hardware from UPMEM, in which case the many filters actually reside on different memory banks. Efficient parallel processing can thus be performed to execute large batches of insertions or lookups. We re-implement this processing using the BPL library.

Limitations encountered Since we execute insertions and lookups in batches, the filters need to exist in memory across several executions. However, the BPL does not support such feature at the moment. Therefore, we had to design an additional class to manage persistent storage and implement it for all architectures supported by the BPL. For UPMEM, the persistent data block is reserved in the MRAM of each DPU. As for Multicore, we allocate all data blocks required, store them in a vector, and adds an index argument to the task so that each instance can retrieve its assigned block.

Benchmarks We consider Bloom filters using 8 hash functions, and a varying size (128 KiB or 1 MiB). For PiM implementations, we allocate 8 ranks, i.e. 512 processing units. One benchmark consists of inserting a varying number of items (10M or 100M), and then querying all the inserted items.

Table 1: Results of the Bloom filters benchmarks

			Multicore time (s)		PiM time (s)	
	Filter size	#items	Standard*	BPL	SDK	BPL
Insertions	128 MiB	10M	0.22	0.56	0.13	0.11
	128 MiB	100M	1.49	4.64	0.66	0.68
	1 GiB	10M	0.39	0.84	0.13	0.11
	1 GiB	100M	2.24	5.56	0.66	0.68
Lookups	128 MiB	10M	0.25	0.41	0.27	0.28
	128 MiB	100M	2.26	3.06	2.13	2.26
	1 GiB	10M	0.38	0.38	0.27	0.28
	1 GiB	100M	2.89	2.94	2.13	2.81

*Uses a single big filter and not the many small filters approach

Results We present the results in Table 1. The Standard Multicore implementation appears faster than the BPL Multicore version, due to being less memory-bound. Indeed, it handles items one at a time instead of regrouping them in batches and processing them altogether. Batching is beneficial with the PiM architecture to minimize the overhead of transferring data between the main memory and the small memory banks, but this purpose is absent on Multicore. Implementing a program on several architectures at once is a great perk of the BPL, but this comes with a caveat: it forces to use the same parallelization scheme for both Multicore and PiM, which is not necessarily the best choice performance-wise. Regarding PiM, the BPL version has similar execution times to the SDK program.

Source code The BPL implementation is publicly available at `https://gitlab.inria.fr/pim/org.pim.bflib-bpl` (GNU AGPLv3). The original Standard Multicore and SDK implementations are available at `https://gitlab.inria.fr/pim/org.pim.bloomfilters` (GNU AGPLv3).

2.2 Mapping based on Hamming distance

Presentation MiMyCS is a software that performs reference-based lossless compression of next-gen sequencing datasets [**?**]. It relies on mapping each read against a known reference and storing the mapping position along with potential substitution errors. The mapping stage is implemented with PiM acceleration by partitioning the reference across memory banks and sending reads to map to only a few well-chosen processing units. We re-implement this mapping procedure using the BPL library.

Limitations encountered We process the reference and need to store the resulting indexing information for all future mapping executions. However, the BPL does not support such feature at the moment. Therefore, we had to design an additional class to manage persistent storage and implement it for all architectures supported by the BPL. For UPMEM, the persistent indexing tables are allocated in the MRAM of each DPU. As for Multicore,

we allocate all tables required, store them in a vector, and adds an index argument to the task so that each instance can retrieve its assigned tables.

PiM specifics To perform the Hamming distance computations as efficiently as possible on the UPMEM architecture, we write manually a few lines of assembly instructions. This part of the task thus uses target-specific code.

Benchmarks We consider a real whole genome sequencing dataset obtained with the Illumina technology, known as accession code SRR14724532 on the European Nucleotide Archive portal. We map this dataset against the human genome GRCh38. For PiM, we use 8 ranks as this is the minimum required to fit the entire reference genome across all memory banks. We report the execution time of the two stages: indexing the reference, and mapping all the reads of the dataset.

Table 2: Results of the mapping benchmark

	Multicore time (s)		PiM time (s)	
	Standard	BPL	SDK	BPL
Indexing reference	33.1	49.4	20.4	34.2
Mapping reads	104.6	142.2	59.1	92.8

Results We present results in Table 2. For both the indexing and the mapping, we see that the BPL introduces a non-negligible overhead and is 63% slower compared to the optimized SDK version. The overhead is for the most part due to the serialization process when retrieving the task results back to the host. Indeed, this involves several memory copies: first on the DPUs, to re-aggregate all the tasklets results into a single buffer, and then on the host, to reformat the transferred buffer into the expected type. All of these copies do not exist when using the SDK.

Source code The BPL implementation is publicly available at `https://gitlab.inria.fr/pim/org.pim.mimycs-bpl` (GNU AGPLv3). The original Standard Multicore and SDK implementations are available at `https://gitlab.inria.fr/pim/org.pim.srm` (GNU AGPLv3).

2.3 Primer alignment with the Smith-Waterman algorithm

Presentation ConCluD is a software that relies on De-Buijn graph partitioning to generate consensus sequences in the context of DNA storage decoding [**?**]. The partitioning step requires to align reads against a set of known primers to find the best fitting one. The published version of ConCluD uses a simple approach based on the Hamming distance. Such computation has already been investigated in Section 2.2 with the MiMyCS software. To explore a different scenario, we thus decide to rather re-implement with the BPL library an older version of ConCluD where the alignment step was originally done with the Smith-Waterman algorithm.

Limitations encountered None: the code can be fully architecture-agnostic.

PiM specifics To perform the alignment computations as efficiently as possible on the UPMEM architecture, we write manually a few lines of assembly code. In particular, we take advantage of the `cmp4` instruction to compare four bytes at once. This part of the task thus uses target-specific code.

Benchmarks We use three real datasets from the experimental evaluation in [**?**], namely Odeuropa, Dunhuang-d0 and HorrorMovie. For PiM, we use 32 ranks.

Results We give results in Table 3. On all datasets, the BPL version is slower than the SDK version regarding the PiM architecture. For the largest dataset, HorrorMovie, it is 34% slower. On top of the BPL overhead, another reason contributes to this observation: we had to split the work statically between tasklets to keep the program entirely architecture-agnostic with regard to the current BPL features. On the other hand, the SDK version uses dynamic scheduling to assign reads to tasklets on the same DPU. The latter is more efficient because the processing time of a read can differ drastically: some reads lead to a perfect alignment very fast, while other require to go through the whole Smith-Waterman matrix or even try several primers.

Regarding the Multicore target, the BPL version is also slower, except for the Dunhuang-d0 dataset where it is faster. The fact that the BPL version processes reads in batches instead of one at a time, combined with the dataset having only one primer, seem to make the BPL version less memory-bound that the Standard program in this case.

Table 3: Results of the primer alignment benchmark

	Multicore time (s)		PiM time (s)	
	Standard	BPL	SDK	BPL
Odeuropa	12.5	16.3	4.7	6.7
Dunhuang-d0	37.0	25.8	12.4	20.6
HorrorMovie	262.4	382.0	92.3	123.9

Source code The BPL implementation is publicly available at `https://gitlab.inria.fr/pim/org.pim.conclud-bpl` (GNU AGPLv3). The original Standard Multicore and SDK implementations are available at `https://gitlab.inria.fr/pim/org.pim.dnarxiv/-/tags/experiments-bioRxiv-preprint` (GNU AGPLv3).

2.4 Sort

Presentation Sort workloads consist of a series of independent applications of small sorting algorithms. In this implementation, the size of the data to be sorted is fixed, as the primary objective is benchmarking against CPU and BPL implementations. Each application sorts a large number of independent arrays.

Limitations encountered The BPL code executed on CPU and PiM is identical; however, the application was originally designed to run on PiM. As a result, the applications are constrained by the maximum array size that can be sorted.

PiM specifics In addition to the BPL version, SDK-based implementations are also provided, as for other workloads. No special instructions are used. The SDK versions are available in two variants: one relying on an automatic cache between MRAM and WRAM (SDK-C), implemented in this work using an RRIP-based policy, and another using explicit manual data transfers(SDK).

Benchmarks Uniformly distributed random arrays of `int32` values are used as input. The number of arrays being sorted depends on the number of ranks (real or emulated), scaling up to 40,960. The array size is limited by the MRAM capacity and the MRAM memory overhead introduced by BPL, and all benchmarks are conducted using the maximum supported size of 131,072 elements. Each benchmark is executed five times, and the reported results correspond to the mean value. To give CPU implementations the best possible performance, applications are compiled with the flags `-march=native` and `-O3`. Additionally, an Intel-optimized version of Quicksort is included (indicated in parentheses).

Results We report the results of the sorting benchmark in Table 4. On both targets, the BPL implementation is consistently slower than the architecture-specific versions. On the PiM architecture, the BPL version exhibits a significant overhead compared to the SDK and SDK-C implementations for all sorting algorithms. For instance, in the case of Quicksort, the BPL version is almost four times slower than the SDK version. This overhead will be further analyzed once execution traces are available.

On the Multicore target, similar trends are observed. The BPL version remains slower than the Standard implementation for all algorithms. The current implementation suffers from unnecessary memory copies, even though the data are already accessible to the CPU cores. Overall, these results highlight the performance cost of maintaining portability and architecture-agnosticism with BPL.

2.5 Sketches comparison

Presentation We present here a benchmark comparing sketches. As a reminder,, Comparing two genomic sequences can therefore be approximated by comparing two sketches, that is, two ordered sequences of integers of the same size N, which can be performed with a complexity of $O(N)$. As a result, a comparison score is produced, making it possible to estimate the similarity between the two genomic sequences.

Limitations encountered For simplicity, the PIM version using the UPMEM SDK only allows certain sizes of the reference and query databases to be used.

Benchmarks For this benchmark, we consider two databases: a reference database containing R sketches, and a query database containing Q sketches. The benchmark consists of performing the $R \cdot Q$ comparisons for each (ref, qry) pair. We will use sketches of $10,000$ integers, each integer encoded on 4 bytes.

Table 4: Aggregated results of sorting benchmark across all ranks and array sizes

			Multicore time (s)		PiM time (s)		
Rank	ARR_SIZE	Algorithm	Standard	BPL	SDK	SDK-C	BPL
1	131072	Quicksort	0.5 (0.1)	0.5	4.9	5.9	8.2
		Heapsort	0.7	0.7	25.4	19.9	25.0
		Radixsort	0.2	0.2	3.2	4.3	6.4
	262144	Quicksort	1.0 (0.2)	1.0	10.1	12.7	17.6
		Heapsort	1.6	1.6	54.1	42.3	52.1
		Radixsort	0.5	0.7	6.5	8.3	NA
	524288	Quicksort	2.1 (0.6)	2.1	21.3	26.8	NA
		Heapsort	3.3	4.3	115.0	89.4	NA
		Radixsort	1.4	1.5	13.0	16.5	NA
10	131072	Quicksort	4.8 (1.0)	4.7	6.5	7.4	12.3
		Heapsort	7.0	6.9	26.9	21.4	28.4
		Radixsort	1.5	1.8	4.8	5.7	10.2
	262144	Quicksort	10.1 (2.5)	9.8	13.2	15.5	24.8
		Heapsort	15.9	15.1	57.2	45.4	59.4
		Radixsort	6.2	5.9	9.6	11.2	NA
	524288	Quicksort	21.0 (6.8)	20.3	27.5	32.9	NA
		Heapsort	48.0	42.4	121.5	95.4	NA
		Radixsort	14.7	13.8	19.0	22.6	NA
20	131072	Quicksort	9.6 (1.8)	9.4	7.2	8.2	14.4
		Heapsort	13.9	13.8	27.7	22.2	30.3
		Radixsort	2.9	3.4	5.6	6.5	12.4
	262144	Quicksort	20.2 (4.9)	19.6	14.4	16.8	29.0
		Heapsort	31.8	30.1	58.6	46.6	63.7
		Radixsort	11.9	11.5	10.8	12.5	NA
	524288	Quicksort	41.9 (13.2)	40.5	29.2	34.5	NA
		Heapsort	95.4	85.5	123.5	97.3	NA
		Radixsort	29.3	27.7	20.8	24.5	NA
30	131072	Quicksort	14.4 (2.8)	14.0	8.2	9.1	16.8
		Heapsort	20.8	20.6	28.5	22.9	32.8
		Radixsort	4.4	5.1	6.5	7.3	14.6
	262144	Quicksort	30.2 (7.2)	29.3	15.8	18.2	34.3
		Heapsort	46.9	45.0	59.8	47.8	68.3
		Radixsort	17.7	17.1	12.2	14.0	NA
	524288	Quicksort	62.0 (19.9)	60.6	31.7	37.2	NA
		Heapsort	102.7	126.6	126.0	99.9	NA
		Radixsort	43.9	40.7	23.8	27.2	NA
40	131072	Quicksort	19.2 (3.7)	18.7	8.8	9.9	19.3
		Heapsort	27.8	27.5	29.2	23.7	35.0
		Radixsort	5.9	7.0	7.2	8.1	17.0
	262144	Quicksort	40.3 (9.5)	39.0	17.3	19.8	38.3
		Heapsort	63.5	60.0	61.1	49.0	73.1
		Radixsort	23.9	22.9	13.7	15.5	NA
	524288	Quicksort	82.6 (26.2)	80.8	34.5	40.3	NA
		Heapsort	136.8	167.9	128.4	102.8	NA
		Radixsort	58.2	55.4	26.6	29.7	NA

We choose to vary the sizes R and Q of the reference and query databases, as well as the number of compute units (threads for multicore and ranks for PIM). In its largest configuration, the reference database contains 245,760 sketches of 10,000 integers of 4 bytes each, totaling over 9 GB of data. For each configuration (reference database size, query database size), the list of scores is computed. A checksum of these scores is then calculated, and we verify that each implementation returns the same checksum for a given configuration,

Table 5: Results of the sketches comparison benchmark

ref	qry	type nbunits	1.MC std	2.MC BPL	3.PIM SDK	duration 4.PIM BPL
24576	**16**	**3**	1.79	2.11	1.31	1.07
		6	0.93	1.10	0.71	0.59
		12	0.48	0.57	0.41	0.37
		24	0.34	0.38	0.26	0.24
	128	**3**	13.50	16.14	9.61	7.64
		6	7.07	8.49	4.88	3.93
		12	3.66	4.37	2.53	2.10
		24	2.33	2.63	1.39	1.19
122880	**16**	**3**	8.53	10.40	6.71	5.38
		6	4.50	5.73	3.44	2.82
		12	2.32	2.77	1.88	1.57
		24	1.56	1.75	1.11	0.95
	128	**3**	67.22	80.53	47.83	37.97
		6	35.42	42.27	24.15	19.26
		12	18.35	21.88	12.29	9.84
		24	9.71	11.66	6.41	5.11
245760	**16**	**3**	17.79	20.77	13.70	10.89
		6	9.33	11.09	6.88	5.68
		12	4.64	5.79	3.92	3.23
		24	2.81	3.42	2.21	1.87
	128	**3**	134.47	161.01	95.88	76.03
		6	70.83	84.50	48.19	38.42
		12	36.70	43.73	24.56	19.55
		24	19.28	22.73	12.63	10.06

regardless of the number of compute units used.

Results We present the results in Table 5. The four implementations tested are:

1. **MC std**: the multicore implementation using standard threads; The BS::thread_pool library is used to simplify thread management
2. **MC BPL**: the multicore implementation using the BPL
3. **PIM SDK**: an implementation directly using the UPMEM SDK; The reading of the reference and query databases is optimized using a local cache to minimize WRAM/M-RAM transfers.
4. **PIM BPL**: the PIM implementation using the BPL

As usual, there is actually a single BPL implementation, executed on both a multicore architecture and a PIM architecture. For this implementation, the reference database is

partitioned into K parts, with each thread or tasklet handling its own partition. The sketches from the query database are thus compared to each of these partitions.

For both PIM versions (SDK UPMEM and BPL) and for a fair comparison, it should be noted that the measured time does not include the duration of DPU allocation/deallocation.

Regarding the two multicore versions, we observe that the BPL version is slightly slower than the standard version, regardless of the configuration. This difference can be explained by two reasons. First, the data partitioning may be more advantageous for the standard version than for the multicore version, resulting in fewer cache misses and therefore fewer memory reloads. Second, the generic nature of the BPL version involves certain design and implementation choices that can sometimes prove less efficient than a more direct implementation.

For the two PIM versions, we observe slightly better performance for the BPL version. This result may seem counterintuitive, since the SDK version is conceptually close to what the BPL can produce in this example, notably with the same data partitioning. The compilation options for both versions are also equivalent for generating the DPU binaries. One possible explanation is that the compiler may be able to better optimize the BPL version compared to the SDK version.

We also observe that scalability is relatively good for all four versions. Since the number of compute units (i.e. threads or ranks) is doubled for each configuration, the execution time should in principle be halved, which is roughly what we observe. For example, none of the versions shows performance stagnation when the number of compute units is doubled.

Finally, for an equivalent number of threads and ranks, we observe that the PIM version is approximately twice as fast as the multithreaded version, whether comparing the two SDK versions or the two PIM versions.

Source code The BPL implementation is publicly available at `https://gitlab.inria.fr/pim/org.pim.bpl/-/tree/main/test/xp/mash` (GNU AGPLv3)

2.6 Sequences comparison

Presentation Sequence comparison is an essential task in genomics, for example to search whether an unknown sequence can be found in a reference database. For instance, the software BLAST has enabled such comparisons for several decades now. Due to the considerable increase in the volume of available sequences, many successors have attempted to improve the underlying algorithm and/or take advantage of the parallelization capabilities of hardware architectures.

We propose here to implement a version of the BLAST algorithm with no gap for the comparison of amino acid sequences on a PIM-type architecture. This version will be called PANG (Protein Alignment with No Gap) and the equivalent command line for the original BLAST would be

```
blastp -query <query.fasta> -subject <db.fasta> -ungapped -out <output>
   -comp_based_stats F -evalue <evalue> -seg yes
```

Limitations encountered None: the BPL code can be fully architecture-agnostic.

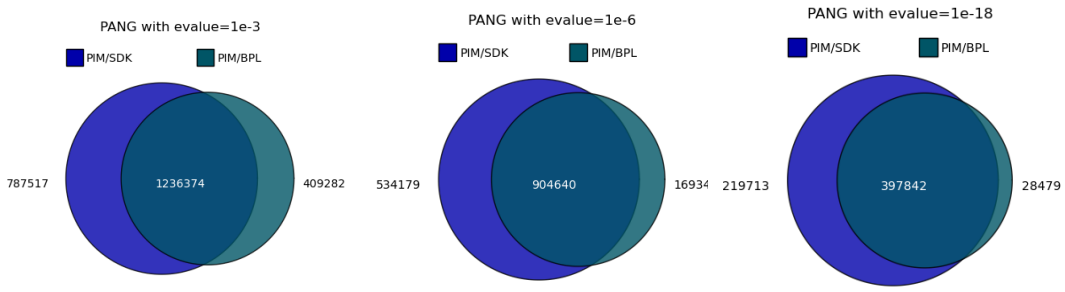


Figure 1: Comparing PANG results

Benchmarks In the following, we will compare a PIM/SDK version of the PANG algorithm with a version implemented using BPL. Note that some heuristics differ slightly, so we will also examine the results themselves in addition to the performance aspects.

We perform a comparison between the well-known SwissProt protein database and a subset of proteins from Arabidopsis thaliana. These two databases contain 563,082 and 15,873 proteins respectively, corresponding to a total of 202,799,066 and 6,788,821 amino acids. For both implementations, we used several different parameter settings, with E-values of 1e-3, 1e-6 and 1e-18 respectively. For the SDK/UPMEM and BPL/UPMEM versions, one rank was used, corresponding to 64 DPUs.

Results We present the results in Table 6 that compares SDK and BPL implementation for an UPMEM architecture.

Table 6: Execution times for Arabidopsis thaliana vs. Swissprot

	PIM/SDK	PIM/BPL
evalue=1e-3	1892	1615
evalue=1e-6	1785	1564
evalue=1e-18	1781	1523

The BPL version exhibits slightly better performance than the version implemented directly with the UPMEM SDK, which may be due to the slightly different heuristics used in the two implementations. To better assess this aspect, the Venn diagrams in figure [1] illustrate the overlap between the [refId, qryId] pairs produced by the two versions. Most of the results are common to both versions, with some results specific to each implementation, likely as a consequence of the different heuristics employed. It can also be observed that for increasingly smaller evalues, the share specific to the BPL version decreases, so that the results obtained by the BPL version are almost included in the results obtained by the SDK version. The fact that the SDK version produces more results may partly explain its slightly longer execution time.

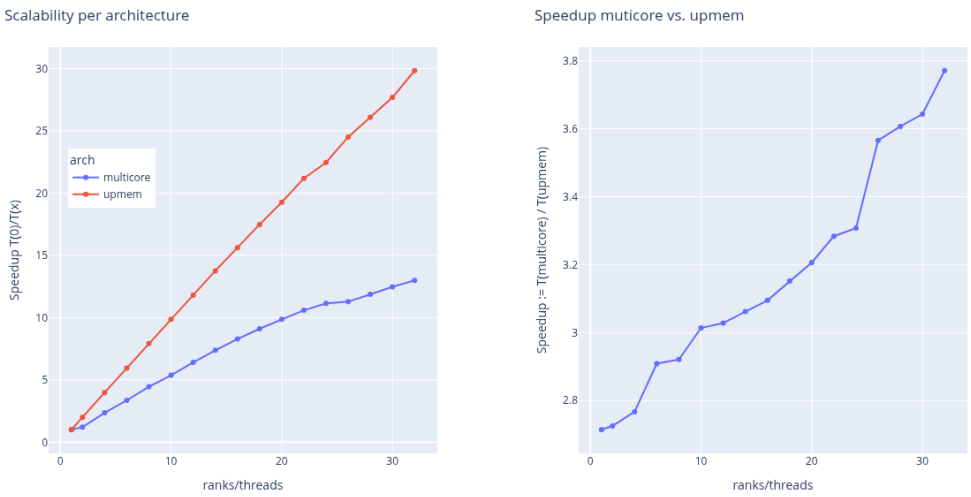


Figure 2: PANG scalability for BPL versions

In conclusion, we present several remarks regarding the scalability of the BPL version on multicore and UPMEM architectures. Figure [2] (left) shows scalability as a function of the number of threads (for multicore) and the number of ranks (for UPMEM). The UPMEM version exhibits good scalability, remaining close to linear, whereas the multicore version shows a noticeable degradation as the number of threads increases. On the right, we analyze the speedup between the multicore and UPMEM versions for an equivalent number of threads/ranks. It can be observed that the UPMEM version benefits from increasing resources, showing improved performance.

Source code The BPL implementation is publicly available at `https://gitlab.inria.fr/pim/org.pim.pang`. The UPMEM SDK implementation is publicly available at `https://gitlab.inria.fr/pim/pang-sdk`.

2.7 Pattern matching

Presentation Searching for a pattern within a text is a common operation in various fields of applications, for instance for natural language processing, but also in genomics with a reduced alphabet containing the four DNA nucleotides. We consider three state-of-the-art pattern matching algorithms: Aho-Corasick (AC), Knuth-Morris-Pratt (KMP) and Boyer-Moore (BM). The Aho-Corasick algorithm [**?**] allows to search for multiple patterns while scanning the text only once, by representing the patterns in a trie and adding failure links to create an automaton. The Knuth-Morris-Pratt algorithm [**?**] uses a pre-computed longest-prefix-suffix array to avoid repeating the same comparisons between the pattern and the text. The Boyer-Moore algorithm [**?**] has a similar approach and skips many positions compared to a naive searching operation. It uses some pre-computed lookup tables to apply a bad-character rule and a good-suffix rule.

Although these algorithms only perform exact matching, they can be adapted to allow for single-letter wildcards to appear in the pattern. We break the input pattern to obtain all slices without any wildcards, and search for the hits of each slice. The resulting positions, offset by the starting index of the slice within the initial pattern, are then reported in a table of counters. When a counter reaches the total amount of slices, this means there is a full match of the initial pattern at this position.

We implement the three algorithms with the BPL library. The pre-processing (i.e. building the automaton or the lookup tables) is performed by the main processor. The reference text is split over all processing units, with some overlapping between the fragments. Each task receives a reference fragment, the pre-computed data, and scans its fragment to apply the algorithm and report the hit positions.

We also write an equivalent implementation that uses the main processor only (denoted Std-Multicore), and another that relies on the UPMEM's SDK (denoted SDK-PiM).

Limitations encountered None: the code can be fully architecture-agnostic.

Benchmarks To compare the performances of a given algorithm depending on the target architecture and implementation, we sample a reference genome and generate datasets of patterns of various characteristics. We set the length L of the patterns to 20 or 50 nucleotides. In a dataset, we aggregate a total N of 1 or 100 patterns. Also, we introduce a number W of single-letter wildcards in the pattern, either 0 or 5. We consider two reference genomes: the human genome GRCh38 [**?**] and the human gut microbiome genome UHGG [**?**].

Results We present the execution time performances of all datasets in Table 7. For the PiM architecture, the first pattern of a dataset is slower than the following because the reference fragment is sent only at the first task execution. The datasets of 100 patterns are thus interesting to measure the full speed the processing can achieve once the reference is set up on all processing units. We see that the BPL-PiM version has then similar performances to the SDK-based implementation. The three pattern matching we consider have a scanning procedure that is fairly simple, the ingenuity lying in the pre-computed data and lookup tables. In this use case where there is little room for advanced instructions and manual optimizations on the DPUs, the BPL exhibits negligible overhead and allows to write easily a standard-like program that performs well.

Source code The implementations are publicly available at `https://gitlab.inria.fr/pim/org.pim.pmatch-bpl` (GNU AGPLv3).

2.8 Regular expressions matching

Presentation In Section 2.7, we explored the matching of simple patterns made of an exact string and possibly a few single-letter wildcards. We now consider a more complex case with the matching of regular expressions. These allow for instance for wildcards of any length, or the repetition of a group of letters. To perform a scan of a reference text and find matching occurrences of an input regular expression, we use the Thompson's construction algorithm [**?**] that relies on a nondeterministic finite automaton (NFA).

Table 7: Results of the pattern matching benchmarks

Reference	L	N	W	Architecture	AC (s)	KMP (s)	BM (s)
GRCh38	20	1	0	Std-Multicore	1.485	0.872	0.361
				BPL-Multicore	1.399	0.765	0.363
				BPL-PiM	0.174	0.255	0.132
				SDK-PiM	0.226	0.258	0.149
GRCh38	50	1	0	Std-Multicore	1.458	0.811	0.277
				BPL-Multicore	1.348	0.71	0.276
				BPL-PiM	0.178	0.22	0.14
				SDK-PiM	0.225	0.227	0.195
GRCh38	20	100	0	Std-Multicore	146.954	81.617	39.711
				BPL-Multicore	137.9	76.47	41.343
				BPL-PiM	11.152	18.752	9.143
				SDK-PiM	10.446	17.875	8.998
GRCh38	50	100	0	Std-Multicore	147.255	82.016	34.009
				BPL-Multicore	139.468	75.447	37.006
				BPL-PiM	10.904	18.092	8.718
				SDK-PiM	10.344	17.24	8.333
GRCh38	50	1	5	Std-Multicore	2.507	5.314	3.852
				BPL-Multicore	2.455	4.778	3.79
				BPL-PiM	1.651	2.304	1.638
				SDK-PiM	1.305	1.986	1.353
GRCh38	50	100	5	Std-Multicore	208.504	510.858	343.22
				BPL-Multicore	199.283	456.502	343.321
				BPL-PiM	83.024	153.039	95.278
				SDK-PiM	79.882	147.772	91.645
UHGG	20	100	0	Std-Multicore	551.344	298.103	142.182
				BPL-Multicore	512.282	267.182	149.261
				BPL-PiM	35.867	57.836	20.858
				SDK-PiM	33.51	55.264	20.301

This automaton has transitions labeled by letters but also ϵ-transitions, meaning a state can have several next possible states, hence its nondeterministic nature. The traversal consists of maintaining a stack of pointers to states in the automaton, and progressing all these pointers at the same time when reading a reference letter, possibly creating new pointers when facing multiple possible transitions. We also store with the pointers an index to remember at which position in the reference the traversal of this pointer has started. If a pointer cannot progress further, it gets deleted. When a pointer however reaches the final state, we record a match at the reference index stored with the pointer. For each letter in the reference, we add a pointer to the starting state of the automaton to the stack, to try to accept the regular expression at every possible position in the reference.

We implement this algorithm with the BPL library. The pre-processing (i.e. building the NFA) is performed by the main processor. The reference text is split over all processing units, with some overlapping between the fragments. Wildcards of any length thus get limited by the overlapping length, which we set to 128 letters. Each task receives a reference fragment, the NFA data, and scans its fragment to apply the algorithm and report the hit positions.

We also write an equivalent implementation that uses the main processor only (denoted Std-Multicore), and another that relies on the UPMEM's SDK (denoted SDK-PiM).

Limitations encountered None: the code can be fully architecture-agnostic.

Benchmarks To compare the performances of a given algorithm depending on the target architecture and implementation, we sample a reference genome and generate datasets of regular expressions of various characteristics. To do so, we pick exact patterns in the genome and alter them to introduce a total of W meta characters such as `*`, `+` or `?`, the original pattern having to be one possible instance of the resulting regular expression. For instance, `CACACTGTTCTGTCCCTCCG` may become `C(AC)*TGT*CTGTCCCTC+G`. We consider W equal to 0, 5 and 10. Also, we denote by L the length of the original exact pattern, either 20 or 50 nucleotides. In a dataset, we aggregate a total N of 10 regular expressions. We consider two reference genomes: the human genome GRCh38 [**?**] and the human gut microbiome genome UHGG [**?**].

Results We present the execution time performances of all datasets in Table 8.

We see that BPL-Multicore exhibits similar execution times to Std-Multicore for all the GRCh38 datasets. The overhead appear however a bit less negligible when matching against the UHGG genome which is four times bigger than GRCh38. On the other hand, BPL-PiM is consistently slightly slower than SDK-PiM.

Source code The implementations are publicly available at `https://gitlab.inria.fr/pim/org.pim.rematch-bpl` (GNU AGPLv3).

Table 8: Results of the regular expressions matching benchmarks

Reference	L	N	M	Architecture	Thompson NFA (s)
GRCh38	20	10	0	Std-Multicore	16.894
				BPL-Multicore	17.759
				BPL-PiM	5.068
				SDK-PiM	4.202
GRCh38	50	10	0	Std-Multicore	17.282
				BPL-Multicore	18.042
				BPL-PiM	5.097
				SDK-PiM	4.217
GRCh38	20	10	5	Std-Multicore	17.733
				BPL-Multicore	18.602
				BPL-PiM	4.890
				SDK-PiM	3.955
GRCh38	50	10	5	Std-Multicore	20.097
				BPL-Multicore	20.953
				BPL-PiM	6.142
				SDK-PiM	4.644
GRCh38	50	10	10	Std-Multicore	19.662
				BPL-Multicore	20.572
				BPL-PiM	6.603
				SDK-PiM	5.095
UHGG	50	10	10	Std-Multicore	83.887
				BPL-Multicore	87.293
				BPL-PiM	29.583
				SDK-PiM	25.709

References

[1] Florestan De Moor, Meven Mognol, Charles Deltel, Erwan Drezen, Julien Legriel, and Dominique Lavenier. Mimycs: A processing-in-memory read mapper for compressing next-gen sequencing datasets. In 2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM), pages 6716–6723, 2024.

[2] Florestan De Moor, Olivier Boullé, and Dominique Lavenier. De-bruijn graph partitioning for scalable and accurate dna storage processing. Bioinformatics, 41(11):btaf618, 11 2025.

[3] Alfred V. Aho and Margaret J. Corasick. Efficient string matching: an aid to bibliographic search. Commun. ACM, 18(6):333–340, June 1975.

[4] Donald E. Knuth, James H. Morris, Jr., and Vaughan R. Pratt. Fast pattern matching in strings. SIAM Journal on Computing, 6(2):323–350, 1977.

[5] Robert S. Boyer and J. Strother Moore. A fast string searching algorithm. Commun. ACM, 20(10):762–772, October 1977.

[6] 1000 Genomes Project Consortium, Gonçalo R. Abecasis, David Altshuler, Adam Auton, Lisa D. Brooks, Richard M. Durbin, Richard A. Gibbs, Matt E. Hurles, and Gil A. McVean. A map of human genome variation from population-scale sequencing. Nature, 467(7319):1061–1073, October 2010.

[7] Alexandre Almeida, Stephen Nayfach, Miguel Boland, Francesco Strozzi, Martin Beracochea, Zhou Jason Shi, Katherine S. Pollard, Ekaterina Sakharova, Donovan H. Parks, Philip Hugenholtz, Nicola Segata, Nikos C. Kyrpides, and Robert D. Finn. A unified catalog of 204,938 reference genomes from the human gut microbiome. Nature Biotechnology, 39(1):105–114, January 2021.

[8] Ken Thompson. Programming techniques: Regular expression search algorithm. Commun. ACM, 11(6):419–422, June 1968.